Railway Scheduling

Hoang Long Nguyen

University of Potsdam, Germany
Bachelor in Informatik/Computational Science
Matrikelnummer: 808101
nguyen7@uni-potsdam.de

Abstract. In this project we utilized the programming language Answer Set Programming (ASP) to simulate train scheduling within the Flatland framework. The goal was to solve the Vehicle Re-Scheduling Problem (VRSP) which deals with reallocating vehicles when a previously planned trip is disrupted and requires dynamic adjustments to schedules. For simplification train movements and tracks were simulated in a 2D grid. At the start of the project train movements and tracks were simulated in ASP without disruptions using rules and constraints. We then extended the code with the concept of Multi-Shot Solving to avoid requiring the user to specify a maximum number of time steps (the horizon) to find a solution for a Flatland problem. Multi-Shot Solving addresses the problem by initially starting with the smallest possible horizon; if no solution is found, the horizon is expanded the program is re-grounded with the new time step and solved again. This eliminates the need for manual horizon adjustments. By integrating the Multi-Shot Solving concept the code became not only more efficient but also allowed for the consideration of malfunctions. To achieve this we use the concept of the Temporal Plan Graph (TPG) which accounts for two types of temporal dependencies: firstly the order in which an individual agent enters locations and secondly, the order in which different agents must enter the same locations to avoid collisions. To solve the VRSP problem we generate an initial plan for a Flatland problem using Multi-Shot Solving and subsequently execute it introducing malfunctions during execution and adjusting train movements to these malfunctions with the help of the TPG.

1 Introduction

1.1 Background and motivation

The Swiss Federal Railways (SBB) operate the most heavily trafficked rail network in the world with over 10,000 trains being routed through more than 13,000 switches and controlled by over 32,000 signals each day. Every day, SBB transports around 1.2 million passengers and 50% of all goods in Switzerland. To meet the increasing transportation demand in the future a capacity increase of about 30% is required. One way to achieve this capacity increase is through the optimization of schedules. [1] The Flatland problem aims to address the Vehicle

Re-Scheduling Problem (VRSP). The VRSP occurs when a previously planned train journey need to be reorganized due to disruptions. Such disruptions can include accidents, medical emergencies or vehicle breakdowns. The VRSP represents a dynamic version of the classical Vehicle Scheduling Problem (VSP). The Flatland problem simulates this scenario in a simplified grid environment that models train movements and disruptions. This environment allows for rapid experimentation and testing of solutions. [1]

1.2 Objectives

In this project, we aim to solve and simulate the Vehicle Re-Scheduling Problem (VRSP) using the Answer Set Programming (ASP) language. ASP is now an established approach for declarative problem solving and is gaining increasing popularity in both academia and industry. The application of ASP extend beyond Artificial Intelligence and Computer Science. One reason for ASP's popularity is that it offers an integrated modeling, grounding, and solving paradigm, making ASP systems accessible even to non-experts. [3]

The project is structured into 4 main sections. In the first section we will focus on the railway environment for the ASP code, utilizing the concepts of the Flatland environment. The second section focuses on simulating train movements without disruptions (malfunctions), where the railway traffic is described by rules and constraints in ASP. The third section addresses the problem of manually setting the maximum time steps which is resolved using the concept of multishot solving. In the fourth section we tackle the actual VRSP problem. Here, malfunctions are introduced into our simulation and the problem is solved using the concept of the Temporal Plan Graph (TPG).

2 Enviroment

For the implementation, we assumed a global observation where all information about the agents (trains) is available. For simplicity we assumed that all trains move at the same speed and advance by exactly one cell per time step. Additionally we used the following predefined fact formats:

Cells: `cell((Y,X), Type)`
Startposition: `start(agent(A), (Y,X), EarliestDeparture, dir(D))`
Destinationpositon: `end(agent(A), (Y,X), LatestArrival)`
Actions: `action(agent(A), Action, Timestep)`

Here, (Y,X) represents the position of the cell, and Type refers to the type of track. agent(A) stands for the agent or train A, where A is an integer. EarliestDeparture indicates the earliest possible time at which a train can leave its starting point. Before this time the train is not part of the railway system and is thus ignored by other trains. dir(D) represents the direction the train is facing with D corresponding to the cardinal directions "east," "north," "west," and

"south." LatestArrival specifies the latest time by which a train must reach its destination. The train disappears from the railway system once it reaches its destination. Action describes the movement of the train which can be one of the following: move_forward, move_left move_right, or wait. move_forward is the default movement, meaning that if a train has only one direction available for movement it is considered a move_forward action even if the train changes its direction in the process.

3 Implementation

3.1 Train movement without malfunctions

After explaining the predefined environment of the problem we now aim to define the rules and constraints in ASP to model the movement of the trains. First we solved simple environments and based on this we generalized the code so that it also works for more complex scenarios. In lines 1 and 2 the horizon is defined specifying when the trrains must reach their target position as well as the cardinal directions. The possible movements of the agents, which only represent the environment's options and not the actual movements of the trains are defined in line 3. In lines 4 and 5, the offsets needed to calculate the new positions of the trains are define along with the direction changes for deadends. The tracks themselves including their type the direction the agent is facing and the allowed movements are described in lines 7-9. Tracks who are switches and deadends are marked in lines 11 and 12. The schedule of an agent is defined in lines 14 and 15 which specifies when a train can depart at the earliest and when it must arrive at the latest. Once an agent reaches its goal is also marked in these lines. The selection of a single movement for each agent at any given time is determined in line 18 as long as the train has not yet reached its target and is within his schedule. The actual movrments of the agents are defined between lines 20-27 as action facts based on the movements defined in move(), the current state and whether the train is on a switch. The initial state of each agent is set in line 29 while the calculation of the agent's state for the next time step is done in lines 30-38. Here a distinction is made between trains that are at a deadend and those that are not. In lines 40 and 41 a constraint rule is used to eliminate movements that deviate from the predefined tracks. So basically we let the agent make a random move at every timestep, then check if the tracks allow the move and translate it into an action. The check to determine if the agents have reached their target is performed in line 42. Finally the constraints in lines 43-45 ensure that agents are only on valid cells no two agents are on the same position at the same time and collisions are avoided.

```
1 time(0..k).
2 dir(e). dir(w). dir(n). dir(s).
3 m(forward).  m(left).  m(right). m(wait).
4 calc_newDir(s,forward,s,( 1, 0)).
5 dir_rev(n,s).
6
```

```
7  track(32800,s,forward).
8  track(32800,n,forward).
9  track(Type,D,wait) :- track(Type,_,_), dir(D).
10
11 switch(38433, n). switch(38433, e).
12 deadend(8192).
13
14 [...]
15
16 schedule(A, EarliestDeparture, LatestArrival):- start(agent(A),_,
       EarliestDeparture,_), end(agent(A),_,LatestArrival).
17 finished(A,T) :- state(agent(A),(Y,X),_,T), end(agent(A),(Y,X),_).
18
19
20 { move(A,M,T) : m(M) } = 1 :- state(agent(A),_,_,T), not finished(A,T),
       schedule(A, EarliestDeparture, LatestArrival), EarliestDeparture <= T
        <= LatestArrival.
21
22 action(agent(A), wait, T) :- move(A,wait,T).
23 action(agent(A), move_left, T) :- move(A,left,T),
24    state(agent(A),(Y,X),D,T), cell((Y,X),Type), switch(Type,D).
25 action(agent(A), move_right, T) :- move(A,right,T),
26    state(agent(A),(Y,X),D,T), cell((Y,X),Type), switch(Type,D).
27 action(agent(A), move_forward, T) :- move(A,M,T),
28    not action(agent(A), move_left, T),
29    not action(agent(A), move_right, T),action(agent(A), wait, T).
30
31 state(agent(A),(Y,X),D,EarliestDeparture) :- start(agent(A), (Y,X),
       EarliestDeparture, dir(D)).
32 state(agent(A),(Y_old+Y,X_old+X),D,T) :-
33    state(agent(A),(Y_old,X_old),D_old,T-1), move(A,M,T-1),
34    calc_newDir(D_old,M,D,(Y,X)), cell((Y_old,X_old),Type),
35    not deadend(Type),time(T).
36 state(agent(A),(Y_old+Y,X_old+X),D,T) :-
37    state(agent(A),(Y_old,X_old),D_old,T-1),
38    move(A,M,T-1), cell((Y_old,X_old),Type),
39    deadend(Type), dir_rev(D_old,D_rev),
40    calc_newDir(D_rev,M,D,(Y,X)), time(T).
41
42 :- move(A,M,T), state(agent(A),(Y,X),D,T), cell((Y,X),Type),
43    not track(Type,D,M).
44 :- state(agent(A),_,_,_), not finished(A,_).
45 :- state(_,(Y,X),_,_), not cell((Y,X),_).
46 :- state(agent(A),(Y,X),_,T), state(agent(B),(Y,X),_,T), A!=B.
47 :- state(agent(A),(Y,X),_,T), state(agent(B),(Y,X),_,T-1), A!=B.
```

Listing 1.1. Shortened code from singleshot encoding (rs_singleshot.lp)

3.2 Incremental solving: multi-shot solving

In Single-Shot Encoding, we currently face the issue that we have to specify the horizon in every execution. However, in a real situation this horizon is unknown. To avoid manually increasing the horizon each time we cannot find a solution, we use the concept of Multi-Shot Solving. Multi-Shot Solving allows us to solve continuously changing logic programs. In Multi-Shot Solving, the Flatland problem is first grounded and attempted to be solved for the smallest horizon. If no solution exists the horizon is incremented and grounded again for the new time step. This adds new information to the program and attempts to solve the Flatland problem once more. This way we do not need to re-ground and re-solve the entire problem with each increment of the horizon.

Incremental solving divides the program into three parts: base, step, and check, and uses external atoms of the form query(t). All three parts are marked in Clingo with #program declarations. The base part describes static information that does not change over time, making it independent of the step parameter t. The step part depends on the step parameter t and receives new information that is contingent on t. We now use this structure to adapt the encoding from Listing 1.1 to an incremental version in Listing 1.2.

```
1  #program base.
2  m(forward).  m(left).  m(right). m(wait).
3  dir(e). dir(w). dir(n). dir(s).
4  calc_newDir(s,forward,s,(1,0)).
5  dir_rev(n,s).
6
7  track(32800,s,forward).
8  track(32800,n,forward).
9  track(Type,D,wait) :- track(Type,_,_), dir(D).
10
11 switch(38433, n). switch(38433, e).
12 deadend(8192).
13
14 [...]
15
16 schedule(A, EarliestDeparture, LatestArrival):- start(agent(A),_,
       EarliestDeparture,_), end(agent(A),_,LatestArrival).
17
18 state(agent(A),(Y,X),D,EarliestDeparture) :-
19    start(agent(A), (Y,X), EarliestDeparture, dir(D)).
20
21 #program check(t).
22 :- {finished(A,0..t)}=0, query(t), end(agent(A),_,_).
23 :- {finished(A,0..LatestArrival)}=0, end(agent(A),_,LatestArrival), query
       (t).
24
25 #program step(t).
26 { move(A,M,t-1) : m(M) } =1 :-
27    state(agent(A),_,_,t-1),
```

```
28      schedule(A, EarliestDeparture, LatestArrival),
29      EarliestDeparture <= t <= LatestArrival.
30
31  action(agent(A), wait, t-1) :- move(A,wait,t-1).
32  action(agent(A), move_forward, t-1) :- move(A,M,t-1), not action(agent(A)
        , move_left, t-1), not action(agent(A), move_right, t-1), not action(
        agent(A), wait, t-1).
33  action(agent(A), move_left, t-1) :- move(A,left,t-1), state(agent(A),(Y,X
        ),D,t-1), cell((Y,X),Type), switch(Type,D).
34  action(agent(A), move_right, t-1) :- move(A,right,t-1), state(agent(A),(Y
        ,X),D,t-1), cell((Y,X),Type), switch(Type,D).
35
36  state(agent(A),(Y_old+Y,X_old+X),D,t) :-
37      state(agent(A),(Y_old,X_old),D_old,t-1),
38      move(A,M,t-1), calc_newDir(D_old,M,D,(Y,X)),
39      cell((Y_old,X_old),Type), not deadend(Type).
40  state(agent(A),(Y_old+Y,X_old+X),D,t) :-
41      state(agent(A),(Y_old,X_old),D_old,t-1),
42      move(A,M,t-1), cell((Y_old,X_old),Type),
43      deadend(Type), dir_rev(D_old,D_rev),
44      calc_newDir(D_rev,M,D,(Y,X)).
45
46  finished(A,t) :- state(agent(A),(Y,X),_,t), end(agent(A),(Y,X),_).
47
48  :- move(A,M,t-1), state(agent(A),(Y,X),D,t-1), cell((Y,X), Type), not
        track(Type,D,M).
49  :- state(agent(A),(Y,X),_,t), state(agent(B),(Y,X),_,t), A!=B.
50  :- state(agent(A),(Y1,X1),_,t-1), state(agent(B),(Y2,X2),_,t-1),
51      state(agent(A),(Y2,X2),_,t), state(agent(B),(Y1,X1),_,t),A!=B.
52  :- state(agent(A),(Y,X),_,t), state(agent(B),(Y,X),_,t-1), A!=B.
53  :- state(_,(Y,X),_,_), not cell((Y,X),_).
```

Listing 1.2. Shortened code from multishot encoding (rs_multishot.lp)

To adapt our encoding from Listing 1.1 to Listing 1.2 we divide the code into the parts base, check(t), and step(t). The base part includes all rules and atoms that are independent of the time parameter t and represent static knowledge. This includes the definitions of the tracks the initialization of the initial state, as well as helper atoms for determining the new direction and position. The step part includes all state transitions, the determination of valid movements and the track constraints described in lines 25-53. The query in the check part is formulated in such a way that it depends on the external atom query(t), which determines whether the query is true or not. The external atom is added to our encoding through a already given Python code, as shown in Listing 1.3.

```
1  import sys
2  from typing import cast, Any, Callable, Optional, Sequence
3  from clingo.application import clingo_main, Application,
       ApplicationOptions
4  from clingo.control import Control
```

```
from clingo.solving import SolveResult
from clingo.symbol import Function, Number

class IncConfig:
    def __init__(self):
        self.imin = 0
        self.imax = None
        self.istop = "SAT"

[...]

class IncApp(Application):
    program_name: str = "inc-railway"
    version: str = "1.0"
    _conf: IncConfig

    def __init__(self):
        self._conf = IncConfig()

    def main(self, ctl: Control, files: Sequence[str]):
        if not files:
            files = ["-"]
        for file_ in files:
            ctl.load(file_)
        ctl.add("check", ["t"], "#external␣query(t).")
        conf = self._conf
        step = 0
        ret: Optional[SolveResult] = None

        while ((conf.imax is None or step < conf.imax) and
               (ret is None or step < conf.imin or (
                   (conf.istop == "SAT" and not ret.satisfiable) or
                   (conf.istop == "UNSAT" and not ret.unsatisfiable) or
                   (conf.istop == "UNKNOWN" and not ret.unknown)))):
            parts = []
            parts.append(("check", [Number(step)]))
            if step > 0:
                ctl.release_external(Function("query", [Number(step - 1)]))
                parts.append(("step", [Number(step)]))
            else:
                parts.append(("base", []))
            ctl.ground(parts)
            ctl.assign_external(Function("query", [Number(step)]), True)
            ret, step = ctl.solve(), step + 1
        print("timesteps:␣",step-1)

clingo_main(IncApp(), sys.argv[1:])
```

Listing 1.3. Shortened code from multishot encoding (rs_multishot.lp)

The Python code from Listing X is used to incrementally ground and solve the encoding from Listing X for a Flatland problem. In this Python code, we use several variables. imin and imax specify the minimum and maximum number of iterations, respectively. istop represents the stopping criterion, which in our case is "SAT," standing for "satisfiable." imin, imax, and istop are user-defined variables initialized in the IncConfig class at line 8. Another important variable is step, which represents the current time step. step is incremented each time no solution is found for the current time step. The variable ret stores and outputs the solution set. The main program consists of the while loop in line 34. The variable part is a list that stores the subprogram for each time step along with its current step value. For example, the first iteration stores and grounds the base program and check(0) because the step value is 0. Additionally, query(0) is declared and initially set to false, later changed to true to activate the query. In line 48, the solve function checks whether the stopping criterion has been met or not, and then step is incremented. This means that as long as the stopping criterion is not met the variable step takes the current iteration value and grounds the subprograms step(t) and check(t). This way, we are able to solve a Flatland problem incrementally.

3.3 Malfunctions and TPG

It does not reflect reality to consider malfunctions and delays as fixed facts during the creation of the Flatland plan. These issues occur randomly during the execution of the plan and are not known during the planning phase. With the multi-shot solving approach we can not only generate a plan for the Flatland problem but also execute the plan and insert delays when necessary. This corresponds to actual conditions. Additionally we utilize the concept of the Temporal Plan Graph (TPG) as described by [Honig et al.]. TPG is a data structure that manages temporal dependencies between agents as they move through different locations. Two types of temporal constraints are considered: 1. Each agent must enter positions in the order they appear in the original plan and 2. At any position entered by two agents they must follow the same order as in the original plan. [2] We intend to apply these types of temporal constraints to our Flatland problem. This means we will first generate a plan for the Flatland problem without considering malfunctions and then execute this plan through multi-shot solving where malfunctions may occur and are handled using TPG. For generating the initial plan we simply use the encoding from version 3.2 but rename the state facts to basestate. Additionally, we modify the Python script so that it saves the basestate facts in a new file named initplan.lp. This gives us a plan that corresponds to the original plan. After generating the initial plan we use the encodings from 1.4 and 1.5 to execute the plan. In the base part we only need the facts that mark the switches as well as facts that allow us to recognize the rotations an agent has made to determine if it has performed a left or right movement as shown in lines 2 to 7. In line 13 we initialize the state at time 0. Finally we use the new fact tpgorder in lines 9 to 11. This checks if there are coordinates where two or more agents pass through in the original plan and

counts using the count function all agents that passed this position before the current agent. This way we know the order in which agents enter that position. The step part contains all action facts in lines 23 to 26 which depend on consecutive states, the direction before and after and the track types. In line 28, we use a choice rule that states that if a delay occurs in move B it may or may not cause delays in other moves. In line 30 the state is generated when no delay has occurred in a previous move. To generate the state when a train has experienced a delay we use the facts activeDelay and oldDelay in lines 32 and 36. These facts sum up all completed delays and calculate the time that has already passed for active delays. This allows us in lines 40 to 43 to track the new state of a train based on the basestate. Additionally we determine the order in line 45 similar to tpgorder when multiple agents enter the same position. Finally we ensure in lines 47 and 48 that trains do not collide. In this encoding we use two check parts in lines 18 and 22. Firstwe check whether the order at conflict positions has been respected and then in the second check we verify if the agent has reached its goal on time. This distinction is important because we need to differentiate between an invalid solution due to a violation of the TPG order and a violation of the goal constraints.

```
1  #program base.
2  switch(37408, n).
3  [...]
4
5  rotationLeft(n,w).
6  rotationRight(w,n).
7  [...]
8
9  tpgorder(agent(A),(Y,X),Order):- basestate(agent(A),(Y,X),_,T1),
10    basestate(agent(B),(Y,X),_,T2), A != B,
11    Order = #count{C,T : basestate(agent(C),(Y,X),_,T), T<T1, C != A}.
12
13 state(agent(A),Pos,D,0):- basestate(agent(A),Pos,D,0).
14
15 #program check(t1).
16 :- curOrder(agent(A),(Y,X),Order,t1), not tpgorder(agent(A),(Y,X),Order),
       query1(t1).
17
18 #program check(t2).
19 :- basestate(agent(A),Pos,_,T1), {state(agent(A),Pos,_,T2):T1<=T2}=0,
       query2(t2).
20 :- end(agent(A),(Y,X),latestArrival), state(agent(A),(Y,X),T), T>
       latestArrival.
21
22 #program step(t).
23 action(agent(A), wait, t-1):- state(agent(A),(Y,X),D,t), state(agent(A),(
       Y,X),D,t-1).
24 action(agent(A), move_forward, t-1):- state(agent(A),(Y1,X1),D,t-1),
       state(agent(A),(Y2,X2),D,t),(Y1,X1)!=(Y2,X2).
```

```
25 action(agent(A), move_left, t-1):- state(agent(A),(Y1,X1),D1,t-1), state(
       agent(A),(Y2,X2),D2,t), cell((Y1,X1),Type), switch(Type,D1),
       rotationLeft(D1,D2).
26 action(agent(A), move_right, t-1):- state(agent(A),(Y1,X1),D1,t-1), state
       (agent(A),(Y2,X2),D2,t), cell((Y1,X1),Type), switch(Type,D1),
       rotationRight(D1,D2).
27
28 0{delay(agent(A),dur(1),t)}1 :- delay(agent(B),dur(Dur1),T1), state(agent
       (A),_,_,t), A!=B, T1+Dur1 >= t.
29
30 state(agent(A),(Y,X),D,t):- basestate(agent(A),(Y,X),D,t), {delay(agent(A
       ),_,0..t)}=0.
31
32 activeDelay(agent(A), Active,t) :-
33     Active = #sum{t-T : delay(agent(A), dur(D), T), T+D>t, t>=T},
34     basestate(agent(A),_,_,_).
35
36 oldDelay(agent(A), Old,t) :-
37     Old = #sum{D,T: delay(agent(A), dur(D), T), T+D<=t},
38     basestate(agent(A),_,_,_).
39
40 state(agent(A),(Y,X),Dir,t):-
41     basestate(agent(A),(Y,X),Dir,t-Old-Active),
42     oldDelay(agent(A), Old,t),
43     activeDelay(agent(A), Active,t).
44
45 curOrder(agent(A),(Y,X),Order,t) :- state(agent(A),(Y,X),_,t),basestate(
       agent(B),(Y,X),_,T2), A !=B, Order = #count{C,T : basestate(agent(C)
       ,(Y,X),_,T), T<t, C != A}.
46
47 :- state(agent(A),(Y,X),_,t), state(agent(B),(Y,X),_,t), A!=B.
48 :- state(agent(A),(Y1,X1),_,t-1), state(agent(B),(Y2,X2),_,t-1),state(
       agent(A),(Y2,X2),_,t), state(agent(B),(Y1,X1),_,t), A!=B.
```

Listing 1.4. Shortened code from encoding (rs_multishot_execution.lp)

After explaining how the ASP code handles delays we now explain how delays are generated in the Python script. In Listing 1.5, lines 15-25 we determine the highest train ID to know how many trains exist. Then in lines 33-37 we initialize new variables: maxDuration, which specifies the maximum duration of a delay probability which defines the likelihood that a train will receive a delay and the lists delayList, delayActiveList, and delayDurationList. delayList stores all delays that occur. delayActiveList contains boolean values indicating whether a train already has a delay or not. This prevents a train from receiving multiple delays simultaneously and ensures that a train must first complete one delay before receiving another. delayDurationList stores the duration of each train's active delay. In the main program there are two for-loops. The first loop iterates over all trains and can generate a delay if the train does not already have an active delay. The second loop deals with active delays. If a train has an active delay the duration in delayDurationList is decremented and then for each

timestep, a delay with a duration of 1 is created and added to the ASP code. In lines 65-78 the queries and subprograms are added. It is important to note that in line 73 query1 checks the TPG constraint and subsequently in line 76 the goal constraint is checked.

```
1  import sys
2  import re
3  import random
4  [...]
5
6  class IncApp(Application):
7      [...]
8
9      def main(self, ctl: Control, files: Sequence[str]):
10         agentlist=[]
11         if not files:
12             files = ["-"]
13         for file_ in files:
14             ctl.load(file_)
15             try:
16                 with open(file_, 'r') as file:
17                     text = file.read()
18                     matches = re.findall('agent\((\d+)\)', text)
19                     for match in matches:
20                         agentID = int(match)
21                         if agentID not in agentlist:
22                             agentlist.append(agentID)
23             except FileNotFoundError:
24                 print(f'{file_} not found.')
25         maxID=(max(agentlist))
26
27         ctl.add("check", ["t1"], "#external query1(t1).")
28         ctl.add("check", ["t2"], "#external query2(t2).")
29
30         conf = self._conf
31         step = 0
32         ret: Optional[SolveResult] = None
33         maxDuration= 5
34         probability = 0.05
35         delayList = []
36         delayActiveList = [False] * (maxID + 1)
37         delayDurationList = [0] * (maxID + 1)
38
39         while ((conf.imax is None or step < conf.imax) and
40                (ret is None or step < conf.imin or (
41                    (conf.istop == "SAT" and not ret.satisfiable) or
42                    (conf.istop == "UNSAT" and not ret.unsatisfiable) or
43                    (conf.istop == "UNKNOWN" and not ret.unknown)))):
44             parts = []
45             parts.append(("check", [Number(step)]))
```

```
46              parts.append(("check2", [Number(step)]))
47
48              for i in range(maxID + 1):
49                  randomNum = random.random()
50                  if randomNum <= probability and not delayActiveList[i]:
51                      duration = random.randint(1, maxDuration)
52                      delayDurationList[i] = duration
53                      delayList.append(f"delay(agent({i}),dur({duration}),{
                            step})")
54                      print(f"A␣delay␣has␣occurred:␣delay(agent({i}),dur({
                            duration}),{step})")
55                      delayActiveList[i] = True
56
57              for i in range(maxID+1):
58                  if delayActiveList[i]:
59                      delayDurationList[i] -= 1
60                      delay = f"delay(agent({i}),dur(1),{step})."
61                      ctl.add("step", ["t"], delay)
62                      if delayDurationList[i] <= 0:
63                          delayActiveList[i] = False
64
65              if step > 0:
66                  ctl.release_external(Function("query1", [Number(step- 1)]))
67                  ctl.release_external(Function("query2", [Number(step- 1)]))
68                  parts.append(("step", [Number(step)]))
69              else:
70                  parts.append(("base", []))
71
72              ctl.ground(parts)
73              ctl.assign_external(Function("query1", [Number(step)]), True)
74              ret = ctl.solve()
75
76              ctl.assign_external(Function("query2", [Number(step)]), True)
77              ret = ctl.solve()
78              step += 1
79  clingo_main(IncApp(), sys.argv[1:])
```

Listing 1.5. Shortened code from encoding (rs_multishot_execution.py)

4 Conclusion

In this project we demonstrated how to address the Flatland problem using ASP. Initially we developed a single-shot approach to simulate basic train movements and ensure that a train can travel from point A to point B. After that we explored the multi-shot solving approach which made it easier for us to find solutions and work more efficiently. Finallly by combining multi-shot solving with the concept of the Temporal Plan Graph (TPG) we were able to incorporate malfunctions into our code and achieved the goal of this project successfully.

References

1. Flatland Problem, https://arxiv.org/abs/2012.05893 [Stand 09.09.2024]
2. Hönig et al., Temporal Plan Graph, https://ojs.aaai.org/index.php/ICAPS/article/view/13796 [Stand 09.09.2024]
3. Kaminski et al., How to build your own ASP-based system ?!, moodle course Railway Scheduling [Stand 09.09.2024]